

EXPERIMENT NO: 1

Date of Performance:

Date of submission :

AIM: To implement Merge Sort using Divide and Conquer approach

PROBLEM DEFINITION: To implement merge sort and perform the analysis.

THEORY:

Divide and conquer technique:

- Breaking the problem into several sub-problems that are similar to the original problem but smaller in size,
- Solve the sub-problem recursively (successively and independently), and then
- Combine these solutions to sub problems to create a solution to the original problem. Merge sort is based on the **divide-and-conquer** paradigm.

Conceptually, a merge sort works as follows

1. Divide the unsorted list into n sublists, each containing 1 element (a list of 1 element is considered sorted).
2. Repeatedly merge sublists to produce new sorted sublists until there is only 1 sublist remaining. This will be the sorted list.

Its worst-case running time has a lower order of growth than insertion sort. Since we are dealing with sub problems, we state each sub problem as sorting a subarray $A[p .. r]$. Initially, $p = 1$ and $r = n$, but these values change as we recurse through subproblems.

To sort $A[p ... r]$:

1. **Divide Step:** If a given array A has zero or one element, simply return; it is already sorted. Otherwise, split $A[p .. r]$ into two sub arrays $A[p .. q]$ and $A[q + 1 .. r]$, each containing about half of the elements of $A[p .. r]$. That is, q is the halfway point of $A[p .. r]$
2. **Conquer Step:** Conquer by recursively sorting the two sub arrays $A[p .. q]$ and $A[q + 1 .. r]$.

3. **Combine Step:** Combine the elements back in $A[p \dots r]$ by merging the two sorted sub arrays $A[p \dots q]$ and $A[q + 1 \dots r]$ into a sorted sequence. To accomplish this step, we will define a procedure MERGE (A, p, q, r). Note that the recursion bottoms out when the subarray has just one element, so that it is trivially sorted.

ALGORITHM:

To sort the entire sequence $A[1 \dots n]$, make the initial call to the procedure MERGE-SORT ($A, 1, n$).

MERGE-SORT (A, p, r)

1. IF $p < r$ // Check for base case
2. THEN $q = \text{FLOOR}[(p + r)/2]$ // Divide step
3. MERGE (A, p, q) // Conquer step.
4. MERGE ($A, q + 1, r$) // Conquer step.
5. MERGE (A, p, q, r) // Conquer step.

Merging:

What remains is the MERGE procedure. The following is the input and output of the MERGE procedure.

Input: Array A and indices p, q, r such that $p \leq q \leq r$ and subarray $A[p \dots q]$ is sorted and subarray $A[q + 1 \dots r]$ is sorted. By restrictions on p, q, r , neither subarray is empty.

Output: The two sub arrays are merged into a single sorted subarray in $A[p \dots r]$.

Algorithm:

MERGE (A, p, q, r)

1. $n1 \leftarrow q - p + 1$
2. $n2 \leftarrow r - q$
3. Create arrays $L[1 \dots n1 + 1]$ and $R[1 \dots n2 + 1]$
4. for $i \leftarrow 1$ to $n1$
5. do $L[i] \leftarrow A[p + i - 1]$
6. For $j \leftarrow 1$ to $n2$
7. do $R[j] \leftarrow A[q + j]$
8. $L[n1 + 1] \leftarrow \infty$
9. $R[n2 + 1] \leftarrow \infty$
10. $i \leftarrow 1$
11. $j \leftarrow 1$
12. for $k \leftarrow p$ to r
13. do if $L[i] \leq R[j]$
14. then $A[k] \leftarrow L[i]$

```

15.         i           i+1
16.    Else A[k]       R[j]
17.         j           j+1

```

We implement it so that it takes $\Theta(n)$ time, where $n = r - p + 1$, which is the number of elements being merged.

INPUT

```

// C program for the implementation of merge sort
#include <stdio.h>
#include <stdlib.h>

// Merges two subarrays of arr[].
// First subarray is arr[left..mid]
// Second subarray is arr[mid+1..right]
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int leftArr[n1], rightArr[n2];

    // Copy data to temporary arrays
    for (i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        rightArr[j] = arr[mid + 1 + j];

    // Merge the temporary arrays back into arr[left..right]
    i = 0;
    j = 0;
    k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        }
        else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of leftArr[], if any
    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }

    // Copy the remaining elements of rightArr[], if any
    while (j < n2) {
        arr[k] = rightArr[j];
        j++;
        k++;
    }

    // The subarray to be sorted is in the index range [left-right]
    void mergeSort(int arr[], int left, int right) {
        if (left < right) {
            // Calculate the midpoint
            int mid = left + (right - left) / 2;

            // Sort first and second halves
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            // Merge the sorted halves
            merge(arr, left, mid, right);
        }
    }

    int main() {
        int arr[] = { 22, 121, 133, 15, 06, 27 };
        int n = sizeof(arr) / sizeof(arr[0]);

        // Sorting arr using mergesort
        mergeSort(arr, 0, n - 1);

        for (int i = 0; i < n; i++)
            printf("%d ", arr[i]);
        return 0;
    }

```

OUTPUT

```

Sorted Array :6
Sorted Array :15
Sorted Array :22
Sorted Array :27
Sorted Array :121
Sorted Array :133

```

CONCLUSION:

Thus we have implemented Merge Sort using Divide and Conquer approach.

Sign and Remark:

R1	R2	R3	R4	Total Marks	Signature
(3)	(5)	(5)	(2)	(15)	